Vidyalaya ERP — QA, Security and Load Test Report

Tested	8–9 November 2026
Build	`main` at `4825e89`, plus the fixes listed in section 2 (in the working tree, not yet deployed)
QA copy	Production build (`next build` + `next start`) on the developer laptop, own database `vidyalaya_qa`, freshly seeded
Live site	https://school-management-module-v3-0.vercel.app: read-only security checks and a capped load test
Roles covered	All 11 school roles, plus parent, student and the platform operator

1. Summary

Functionally the app is in good shape. Every automated suite passes on the QA copy. The only failures are three checks for online payment, which needs a payment gateway that is not configured. **Security is strong at the application level.** The new 510-check security suite found no authorization, injection or session flaw. **The serious problems were around the app, not in it:** vulnerable dependencies, and a live deployment whose functions run on the other side of the world from the database.

Area	Checks	Result
Unit tests (vitest)	431	✓ all pass
API integration: sign-in, roles, family scope, tenant isolation, platform	114	✓ all pass
Permission matrix: 11 roles × every module × view / create / edit / delete	1,170	✓ all pass
Cross-role workflows (admission to leaving, fees, exams, payroll…)	107	104 ✓ · 3 ✗ online payment, gateway not configured (expected)
Feature suites (hostel, live tracking, online exams, documents, alumni…)	293	✓ all pass
Security suite (new: `tests/api/security.ts`)	510	✓ all pass
Browser: every sidebar page, signed in as each of 11 roles	11 roles	✓ no page errors, error screens or failed API calls
Browser: newer screens at 375 px and 1280 px	118 visits	✓ no errors, no horizontal overflow
Browser: platform console, all tabs	4 tabs	✓
Real email through the upgraded mail library	1	✓ delivered to a local SMTP server

Area	Checks	Result
Load test, QA copy (single instance)	10 stages	target met up to about 10 users with no pause between requests; limits in section 5
Load test, live site	4 stages	✗ 3–14 s responses: functions in the wrong region (fix ready)

7 issues were found and fixed in code. All tests pass after the fixes. None of them is live until the changes are pushed to `main`.

---

2. Bugs found and fixed

#	Severity	Finding	Fix	Verified by
1	**Critical**	Next.js 16.3.8 had two remote-code-execution advisories and one denial-of-service advisory (GHSA-2464-fhr4-w9xw, GHSA-673q-mjm6-r5xr, GHSA-p3m5-59mp-jv5r). The image optimiser accepts images from any HTTPS host.	`next` and `eslint-config-next` upgraded to **16.4.5**	full regression: every suite above
2	**Critical** (performance)	Vercel ran the functions in Washington (`iad1`) while the database is in Singapore: `/api/me` 3.8 s, dashboard 12 s, sign-in 11 s, 30–42% time-outs at 10–25 users	`vercel.json` pins functions to Singapore (`sin1`). Applies on the next deploy	the same build next to its database: 26 ms, 124 ms, 0.82 s
3	**High**	`nodemailer` 7.0.13: 20 advisories, including arbitrary file read / SSRF, header injection and address-parser denial of service	upgraded to **10.4.0**. It ships its own types, so `@types/nodemailer` was removed	real SMTP send: correct envelope, From and Subject; email suite
4	**High**	`jspdf` 2.5.2 (critical) and its `dompurify` 2.5.9 (high) were installed but used only by the old `legacy/` folder	removed `jspdf` and `jspdf-autotable`	build and all suites
5	**Medium** (security + support)	Sign-in copied raw database errors into the URL (`/api/auth/error?error=Invalid prisma…`) and told users “Invalid school code, username or password” during an outage	`authorize()` lets only known codes through. Anything else is logged on the server and reported as `UNAVAILABLE` . The form shows “Sign-in is temporarily unavailable”	a server pointed at an unreachable database returns `error=UNAVAILABLE` , logs the real error, and leaks nothing
6	**Medium** (performance)	A cost-12 bcrypt hash ran synchronously when `src/server/crypto.ts` loaded. Almost every API route imports it, so each cold start lost about 0.4 s	precomputed dummy hash	module loads in 30 ms instead of about 440 ms; unknown-user sign-in still costs a full check (471 ms), so timing still does not reveal which usernames exist
7	**Low** (UI)	Greeting read “Good morning, Dr.” and the assistant “Namaste, Dr.!” for “Dr. Anand Swaroop Pathak”	`firstName()` in `src/lib/utils.ts` skips titles (Dr., Mr., Smt., Prof. …)	new unit test

After the fixes, `npm audit --omit=dev` went from **2 critical, 4 high, 4 moderate** to **2 moderate**. Both remaining items are `uuid` inside `exceljs` . The advisory covers `v3` / `v5` / `v6` called with a buffer argument, and `exceljs` only calls `v4()` without one, so it cannot be exploited here.

Not a bug, but fixed for the demo: the income-vs-expense chart showed a loss every month. The seed staffs DEMO like a 500-pupil school but bills 200. The demo top-up now adds monthly development-charge income (see the deployment guide, section 4).

---

3. Functional testing

Every role was tested at three levels: the permission matrix (API), journeys that cross roles (API), and the real browser.

Role	Permission matrix	Browser: every sidebar page	Cross-role journeys
Owner (Super Admin)	✓	✓	admissions, certificates, settings, platform isolation
Admin (Principal)	✓	✓	leave approval, exams, settings, gate, discipline
Teacher	✓	✓	attendance, marks, homework review, notices, messages, online exams
Accountant	✓	✓	fees, receipts and cancellation, payroll, Tally and QuickBooks export
Receptionist	✓	✓	enquiries, admissions pipeline, visitors
Librarian	✓	✓	issue, return, fines
Transport manager	✓	✓	routes, live bus tracking
Store manager	✓	✓	sales, stock, cancellation
Staff	✓	✓	own payslip only
Parent	✓	✓	leave, fees, results, live bus and ETA, messages, polls
Student	✓	✓	homework submission, online exams, results
Platform operator	✓ (locked out of school data)	✓ console tabs	schools, plans, subscriptions, website leads

Journeys covered (each one undoes what it creates): admission → pipeline → admit → logins → leaving; attendance with scope and edit window; leave application → approval → notification; exams → marks (own subject only) → publish → family results; homework → submission → review → resubmission; fees → part payment → receipt → cancellation restores dues; concurrent receipt cancellations; advance fees; imports; payroll; library; store; transport; noticeboard targeting; parent–teacher messages and meetings; behaviour; certificates with public verification; hostel; live tracking; push notifications; languages; gate passes; discipline; report-card remarks and gradebook; online exams (no answer-key leak); polls; discussions; gallery; blog; tasks; documents; reminders and the scheduler; alumni; placements; email; SSO settings; data export and the report builder. One invariant check closes the run: every invoice's paid amount equals its active receipt allocations.

Not covered: online payment end to end. Production builds switch the payment sandbox off by design, and no Razorpay keys are configured, so the server correctly replies “Online payment gateway is not configured” (HTTP 503). Test it on the dev server or with Razorpay test keys before taking payments.

4. Security testing

Checked against the OWASP Top 10 (2021). Active tests ran on the QA copy; only read-only checks touched the live site.

OWASP category	What was tested	Result
A01 Broken access control	All 227 API route files probed anonymously (409 method × route requests); 1,170-check role matrix; another school reading or editing a DEMO student and file by id; teacher hiring an owner; principal granting owner or admin rights; parent reading platform statistics	✓ pass
A02 Cryptographic failures	bcrypt cost 12; AES-256-GCM field encryption; HSTS with preload; TLS to the database (`sslaccept=strict`); production secrets stored as sensitive variables	✓ pass
A03 Injection	No raw SQL built from strings (every raw query is a parameterised template); SQL, path and template probes on list and detail endpoints (no 5xx, nothing internal returned); 8 XSS payloads through the HTML sanitiser; objects where text is expected	✓ pass
A04 Insecure design	Sign-in limited per IP (30 per 15 min) and per account (10 per 15 min); lockout after 5 wrong passwords; every public endpoint rate-limited per IP; honeypot on the lead form	✓ pass. Note: lockout lets someone lock a known username for 15 minutes; this is a common, accepted trade-off
A05 Security misconfiguration	CSP with `frame-ancestors 'none'` , `object-src 'none'` , `base-uri 'self'` ; X-Frame-Options DENY; nosniff; COOP; Referrer-Policy; Permissions-Policy; no X-Powered-By; API responses `no-store` ; source maps and dotfiles not served	✓ pass · info: CSP allows inline scripts, a trade-off documented in `next.config.js`
A06 Vulnerable components	`npm audit` of production dependencies	✗ → ✓ fixed (bugs 1, 3, 4)
A07 Authentication failures	Same error for unknown school, unknown user and wrong password; CSRF token required; cookie HttpOnly + SameSite=Lax (+ Secure on HTTPS); tampered and unsigned (`alg: none`) tokens rejected; cross-origin writes blocked; `callbackUrl` to another site ignored	✓ pass · open: demo accounts with the published password exist on the live site
A08 Integrity failures	Uploads checked by content against an allowlist: HTML, SVG with script and a disguised `.exe` all refused; files served with `sandbox` CSP and nosniff; size limit enforced; students cannot upload staff material	✓ pass
A09 Logging and monitoring	Audit log for sign-in, failed sign-in and lockout; unexpected sign-in errors now logged on the server (bug 5)	✓ pass

OWASP category	What was tested	Result
A10 SSRF	Mail library SSRF fixed (bug 3); the image optimiser accepts any HTTPS host	✓ patched · recommend restricting `images.remotePatterns`

Also checked: no secrets in any tracked file (API keys, private keys, tokens, database URLs with passwords). The live site's headers match the code. Vercel's own DDoS protection challenged the load generator after a burst of about 3,000 requests, which is good protection, but it also caps how hard one machine can test the live site.

Re-run the suite at any time against a server you can write to:

```
TEST_BASE_URL=http://localhost:3100 npx tsx tests/api/security.ts
```

---

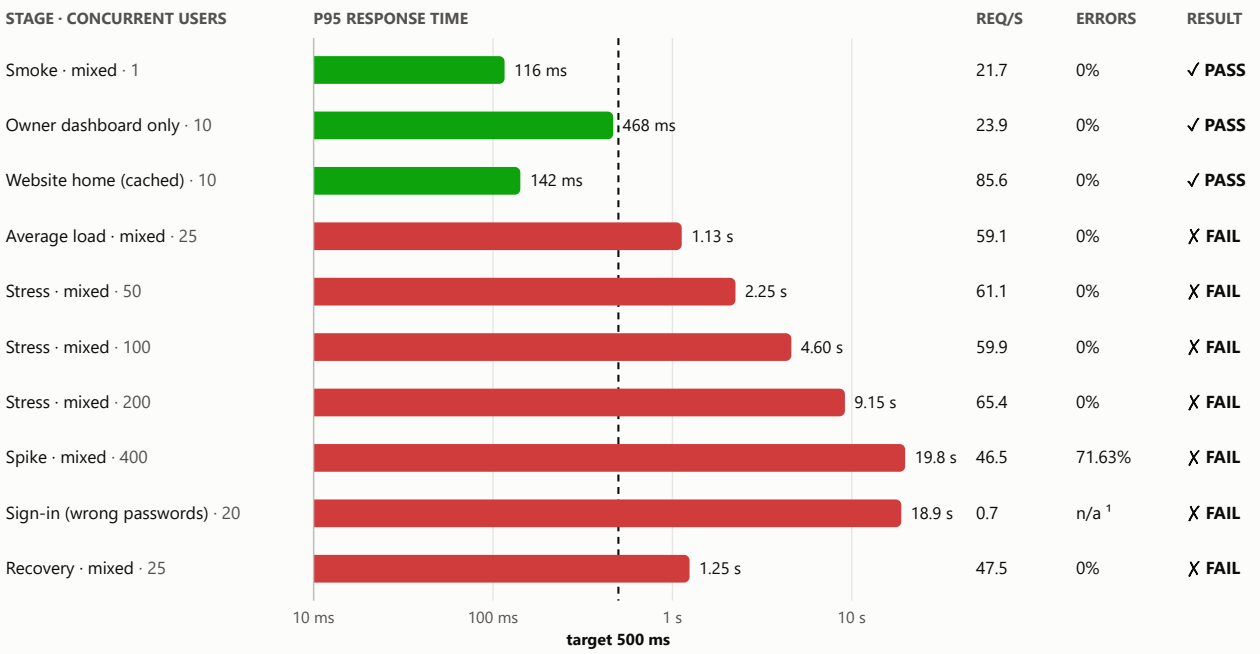
5. Load testing

Method. A staged closed-model test with autocannon, with exact percentiles computed from every response. The stages are smoke, single-endpoint baselines, average load, a stress ramp (50 → 100 → 200 users), a spike (400 users), sign-in, and recovery. The workload is a realistic mix: website home, owner and parent dashboards, profile, student list, notices and sign-in page. Users send requests back to back, with no pause in between, so 25 test users put out roughly as many requests as several hundred real people.
Targets: p95 ≤ 500 ms, p99 ≤ 1.5 s, errors under 1%.

QA copy: one instance, local database

Load test: one app instance on the laptop, local MySQL

p95 response time per stage (log scale) · target p95 ≤ 500 ms, p99 ≤ 1.5 s, errors < 1% · no think time between requests

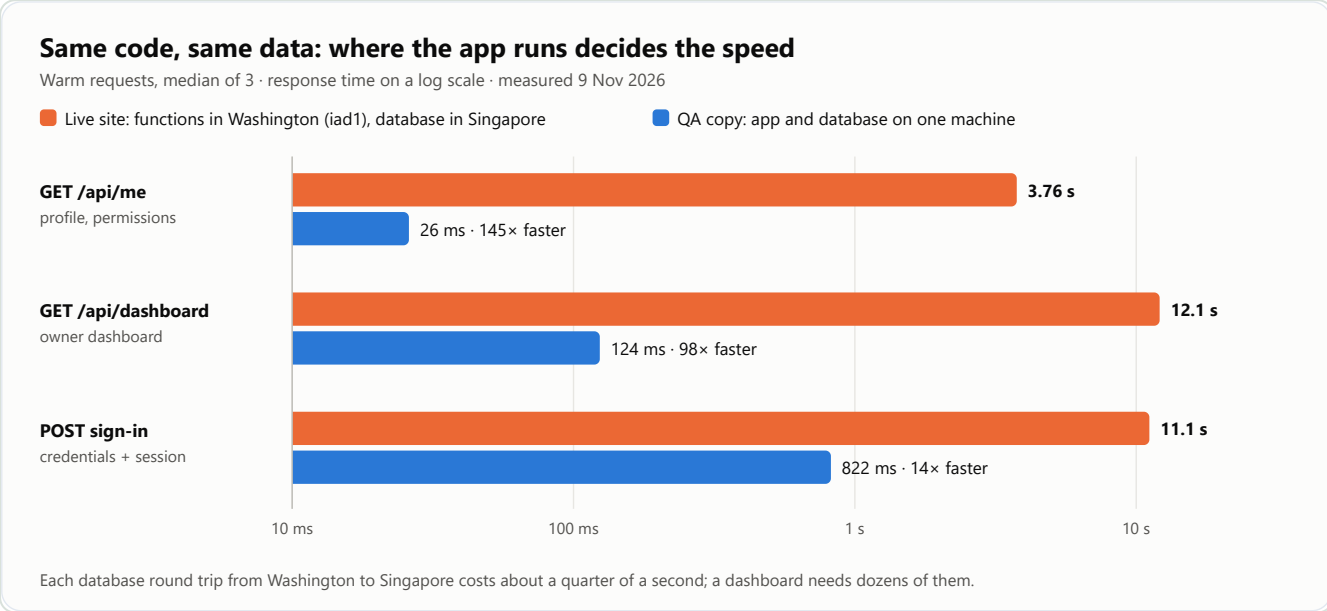


Throughput tops out near 60 requests/s on one Node process; on Vercel each region scales out to more instances.
[1] Sign-in uses wrong passwords, so every response is meant to be 401. It fails on time: one CPU-bound bcrypt hash per attempt.

- **One Node process tops out near 60 requests/s** on the mixed workload, whatever the concurrency. Past that, extra users only add queueing time; that is the single-instance limit, and it's what the failing stages show. With people pausing about 10 s between clicks, 60 requests/s is roughly 600 people actively clicking at once. On Vercel, extra instances start automatically, so this is the limit per instance, not for the whole site.
- **No errors up to 200 back-to-back users.** At 400 the instance runs out and 72% of requests time out after 20 s. It recovers once the spike ends.
- **Sign-in handles only about 0.7 attempts/s per instance** (19 s each with 20 at once). The library is `bcryptjs`, which is pure JavaScript and runs one cost-12 hash at a time: 415 ms each on this laptop, 104 ms at cost 10. A burst of sign-ins, such as a whole class at 9 AM, queues behind it. Recommendation 2 below.

Live site, before the fixes

Stage	Users	p50	p95	Errors
Smoke, mixed	1	3.0 s	14.7 s	0%
Light load, mixed	10	5.6 s	17.5 s	29.8% (time-outs)
Moderate, mixed	25	2.7 s	12.9 s	42.4% (time-outs)
Website home (CDN cache)	25	123 ms	177 ms	blocked by Vercel's DDoS challenge after a burst



The website pages come from Vercel's CDN and are fast. Everything that touches the database is slow, because each query travels Washington ↔ Singapore (bug 2). A query from India takes about 120 ms, and opening a TLS connection about 1.7 s. Inside Singapore, a query takes a few milliseconds.

Not measured yet: the live site after the region fix. Re-run `tests/load/loadtest.mjs` with `PROFILE=prod` after the deploy. Keep it short: the stages are capped, and Vercel challenges a single address that sends a few thousand requests in a burst.

---

6. Recommendations

Priority	Recommendation
1 · now	Deploy the fixes (commit and push to `main`). This applies the Next.js and mail-library security patches, the sign-in error handling, the cold-start fix and the Singapore region. Then re-run the live load test.
2 · now	Change the `platform` password and the DEMO `owner` password on the live site. Remove the sample schools (`npx tsx docs/tools/demo-kpi.ts --remove-samples`) before real customers arrive.
3 · soon	Faster sign-in: replace `bcryptjs` with a native bcrypt (for example `@node-rs/bcrypt`). It reads the same hashes, so nobody's password changes, and it uses all CPU cores. Or lower the cost to 11 for new hashes. This is a security trade-off for you to decide.
4 · soon	Move file uploads to object storage. Vercel cannot keep files written next to the app.
5 · soon	Restrict `images.remotePatterns` in `next.config.js` to the hosts the app actually uses.
6 · later	On Vercel “today” is the UTC day, so dashboards roll over at 05:30 IST. Consider the school's time zone for day boundaries.
7 · later	Re-run this report after each release: the suites above take about 25 minutes in total on the QA copy.

7. How to reproduce

```
 # QA database and production build (bash; PowerShell: $env:NAME = "value")
export DATABASE_URL='mysql://USER:PASS@localhost:3306/vidyalaya_qa'
npx prisma db push --skip-generate && npx tsx prisma/seed.ts && npx tsx docs/tools/demo-kpi.ts
export NEXTAUTH_URL=http://localhost:3100 NEXTAUTH_SECRET=<48 random bytes> DATA_ENCRYPTION_KEY=
<32 random bytes> CRON_SECRET=test-cron-secret-0123456789
npx next build && npx next start -p 3100 &

export TEST_BASE_URL=http://localhost:3100
npx vitest run tests/unit
for s in run rbac-matrix workflows features gap-features security; do npx tsx tests/api/$s.ts;
done
npx playwright test                                                          # 5 roles
E2E_ROLES=owner,reception,librarian,transport,store,staff npx playwright test
node tests/tmp/smoke-gap.cjs
BASE=http://localhost:3100 PROFILE=local node tests/load/loadtest.mjs load.json
```